

Machine Coding Interview Playbook

Questions, Complete Java Solutions & Best Practices
for the 1-Hour Machine Coding Round

- The 60-minute battle plan and what interviewers actually score
 - A beginner-friendly concurrency & locking toolkit (with code)
- 12 classic problems solved end-to-end: Parking Lot, BookMyShow,
 - LRU Cache, Rate Limiter, Splitwise, Snake & Ladder and more
 - Rapid-fire follow-up Q&A, common mistakes, final checklist

All solutions in plain Java (11+). No frameworks required.

Contents

1. How the Machine Coding Round Works (and How to Win It)
2. Universal Best Practices: Design, Clean Code & Patterns
3. The Concurrency & Locking Toolkit (Beginner Friendly)
4. Solved Problems
 1. Parking Lot 2. Movie Ticket Booking (BookMyShow) 3. Thread-Safe LRU Cache
 4. Rate Limiter 5. Flash-Sale Inventory 6. Key-Value Store with TTL
 7. Splitwise 8. Snake & Ladder 9. Vending Machine (State Pattern)
 10. Async Logger 11. Task Scheduler 12. Elevator System
 - + 12 more frequently asked problems with approach hints
5. Rapid-Fire Follow-Up Questions (Concurrency & Java)
6. Common Mistakes That Get Candidates Rejected
7. The Final 60-Minute Checklist

Tip: How to use this playbook: read Sections 1–3 once, then practice each problem by coding it yourself in 45–60 minutes **before** reading the solution. Compare, note the gaps, repeat. Typing the code out beats reading it ten times.

1. How the Machine Coding Round Works (and How to Win It)

A machine coding round gives you a small real-world system — a parking lot, a ticket-booking flow, a cache — and 45–90 minutes to build a **working, extensible, object-oriented program** on your own laptop or a shared editor. There is no database, no REST API, no UI: everything lives in memory, and you demonstrate it through a driver class (`main()`) or simple tests. It is the round where product companies separate people who can *talk* about design from people who can *build* it under time pressure.

What interviewers actually score

Dimension	Weight	What they look for
Working, runnable demo	Highest	Does it compile and satisfy the stated requirements end-to-end? A partial feature set that runs beats a complete design that doesn't.
Object-oriented design	High	Clear entities, single responsibility, interfaces where behavior can vary, easy to extend for the inevitable follow-up (“now add a new vehicle type”).
Code quality	High	Meaningful names, small methods, no duplication, no dead code, consistent structure.
Edge cases & robustness	Medium	Invalid inputs, boundary conditions, custom exceptions — and thread safety when shared state is involved.
Communication	Medium	Clarifying questions up front, thinking aloud, a confident final walkthrough of design decisions and trade-offs.

The 60-minute battle plan

Time is the real opponent. Internalize this split and practice it until it is muscle memory:

Minutes	Phase	What you do
0–7	Clarify & scope	Ask questions (list below), write the agreed requirements as comments at the top of your file, state your assumptions out loud.
7–15	Design skeleton	Create enums, entity classes, interfaces and empty service methods. Say the design in one breath and get a nod from the interviewer before coding bodies.
15–40	Core happy path	Implement the main flows end-to-end. Keep the code compiling every few minutes — never go 15 minutes without a green build.
40–50	Edge cases + concurrency	Input validation, custom exceptions, and locks/atomics wherever state is shared between threads.
50–57	Demo	A <code>main()</code> that prints 3–4 scenarios, including one failure case and (if relevant) one concurrent stress test.
57–60	Walkthrough	Explain the design, name the patterns you used and the trade-offs you made, and say what you'd do with more time.

Questions to always ask in the first 7 minutes

- Which operations are **must-have** vs nice-to-have? (“Should unpark compute the fee, or just free the spot?”)
- **Will this be accessed by multiple threads?** This single question changes your whole design — and asking it is itself a strong signal.
- Is in-memory storage fine? Roughly how many entities should I assume?
- How should I demonstrate it — a driver `main()`, a simple CLI, or unit tests?
- Any hints about what changes later? (“Pricing may change” is code for: use the Strategy pattern.)
- On invalid input, do you prefer exceptions or returned status values?

Tip: Golden rule: **working beats perfect**. Build the simplest version that satisfies the requirements, keep it running at all times, and extend in small safe steps. Interviewers reject beautiful diagrams with broken code every single day.

Jaydip

2. Universal Best Practices: Design, Clean Code & Patterns

2.1 Organize the code like a mini-project

Even in one folder, use packages (or clearly grouped files). It takes 60 seconds and instantly signals maturity:

A layout interviewers recognize immediately

```
machinecoding/  
  models/      -> Vehicle, ParkingSpot, Ticket, Seat, Expense ... (data + tiny logic)  
  enums/       -> VehicleType, SeatStatus, SplitType, LogLevel ...  
  strategies/  -> FeeStrategy, SplitStrategy, AllocationStrategy (interfaces + impls)  
  services/    -> ParkingLot, BookingService, ExpenseService (business logic)  
  exceptions/  -> SpotUnavailableException, InvalidTicketException ...  
  Main.java    -> the demo driver that tells the story
```

2.2 SOLID, translated to machine coding

Principle	One-line meaning	In this playbook
S — Single Responsibility	A class has one reason to change.	ParkingLot allocates spots; HourlyFeeStrategy prices; Ticket just holds data. Pricing changes never touch allocation code.
O — Open/Closed	Add behavior by adding classes, not editing old ones.	New split type in Splitwise = one new SplitStrategy class. Zero edits to ExpenseService.
L — Liskov Substitution	Subtypes must work wherever the parent works.	Any FeeStrategy can replace another without surprising the caller.
I — Interface Segregation	Many small interfaces beat one fat one.	Appender only has append(); it is not forced to implement rotation or formatting.
D — Dependency Inversion	Depend on interfaces, inject implementations.	BookingService receives strategies via constructor — swap them in tests or follow-ups.

2.3 Design patterns that actually show up (and where)

Pattern	Use it when...	Typical problems
Strategy	an algorithm can vary (pricing, splitting, allocation).	Parking fees, Splitwise split types, elevator assignment, rate-limit algorithms
Factory	creation logic depends on a type/enum.	Creating vehicles, chess pieces, notification channels
Singleton	exactly one shared service must exist.	ParkingLot, Logger, configuration (use the holder idiom, Sec 3.11)
State	behavior changes with the object's state.	Vending machine, elevator, order lifecycle
Observer	many parties react to one event.	Order status updates, low-stock alerts, score updates
Chain of Responsibility	a request passes along handlers.	Log levels, approval workflows, ATM cash dispensing (2000/500/100 notes)
Builder	an object has many optional fields.	Complex requests, configurable game/board setup
Repository (idea)	storage should hide behind an interface.	In-memory Map today, real DB tomorrow — callers never know

Watch out: Patterns are seasoning, not the meal. Two well-chosen patterns explained clearly score higher than six interfaces wrapped around a 2-case problem. Name the pattern out loud when you use it — free marks.

2.4 Clean code habits that cost nothing

- Names read like English: tryOccupy(vehicle), not proc(v). Booleans start with is/has/can.

- Methods do one thing and fit on a screen (roughly 15 lines). Extract helpers aggressively.
- No magic numbers: `HOLD_MS = 5 * 60 * 1000` as a named constant, or a constructor parameter.
- Fail fast: validate inputs at the top of public methods and throw a **custom exception** (`InvalidTicketException`) with a helpful message.
- Make fields `private final` wherever possible; expose behavior, not data. Immutable objects are free thread safety (Sec 3).
- Return values instead of printing from services; only `Main` prints. Services stay testable.
- Delete dead code and commented-out experiments before the walkthrough.

2.5 Demo like a professional

The last 10 minutes decide the verdict. Your `main()` should tell a short story: a happy path, a boundary case, a failure case, and — when the problem involves shared state — a concurrent stress test using an `ExecutorService` that proves correctness (see Problem 5 for a template). Narrate each printed line as it appears.

Jaydip

3. The Concurrency & Locking Toolkit (Beginner Friendly)

Most machine coding problems hide the same trap: **one piece of shared mutable state** (a seat, a parking spot, a stock counter) that **two threads touch at the same time**. Handle it well and you jump a level in the interviewer's eyes. This section gives you the complete vocabulary and the exact code idioms — each one reappears inside the solved problems.

Five terms in plain words

- **Thread** — an independent worker executing your code. Two threads can run the same method at the same instant.
- **Race condition** — the final result depends on which thread got there first. Symptoms: double-booked seats, negative stock, lost updates.
- **Critical section** — the few lines that read and then modify shared state; only one thread may be inside at a time.
- **Lock (mutex)** — a gate around a critical section. A thread acquires it, works, releases it; others wait.
- **Atomic operation** — happens as one indivisible step; no thread can observe it half-done.

3.1 The anatomy of a race condition: check-then-act

Almost every interview race is this exact shape — a check followed by an act, with a gap in between:

The bug behind almost every concurrency question

```
// WRONG - looks harmless, oversells under load
if (stock > 0) {           // Thread A checks: stock == 1. Thread B checks: stock == 1.
    stock = stock - 1;    // Both decrement. Final stock == -1. Two items sold, one existed.
}
```

Every tool below is just a different way to make the *check* and the *act* happen as one atomic unit.

3.2 Tool #1 - synchronized (your default)

```
class Counter {
    private int count = 0;

    // Only ONE thread can be inside any synchronized method of this object
    // at a time. The lock is the object itself ("monitor lock").
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }
}

// Prefer a synchronized BLOCK when only a few lines are critical:
public void process(Order order) {
    validate(order);           // safe outside - no shared state
    synchronized (this) {     // keep the locked region as SMALL as possible
        inventory.reserve(order);
    }
    sendEmail(order);          // slow I/O stays OUTSIDE the lock
}
```

- Simple, impossible to forget to unlock, ideal default for machine coding.
- Also guarantees **visibility**: changes made inside the lock are seen by the next thread that takes the lock.
- Rule of thumb: lock the smallest region that covers the check *and* the act; never do I/O while holding a lock.

3.3 Tool #2 - ReentrantLock (when you need more control)

```

class SeatHold {
    private final ReentrantLock lock = new ReentrantLock();

    public boolean tryReserve() throws InterruptedException {
        // tryLock = "attempt for 2s, then give up" - impossible with synchronized.
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                return doReserve();
            } finally {
                lock.unlock();          // ALWAYS unlock in finally - non-negotiable
            }
        }
        return false;                  // couldn't get the lock -> fail gracefully
    }
}

```

Choose `ReentrantLock` over `synchronized` when you need: a timeout (`tryLock`), fairness (new `ReentrantLock(true)` serves waiters in order), interruptible waiting, or one lock guarding code across multiple methods.

3.4 Tool #3 - ReadWriteLock (read-heavy data)

```

class PriceBoard {
    private final ReadWriteLock rw = new ReentrantReadWriteLock();
    private final Map<String, Double> prices = new HashMap<>();

    public Double get(String item) {
        rw.readLock().lock();          // MANY readers may hold this together
        try { return prices.get(item); }
        finally { rw.readLock().unlock(); }
    }
    public void update(String item, double price) {
        rw.writeLock().lock();          // writer waits for exclusive access
        try { prices.put(item, price); }
        finally { rw.writeLock().unlock(); }
    }
}

```

Use when reads massively outnumber writes (price boards, config, catalogs). Readers proceed in parallel; only writes serialize.

3.5 Tool #4 - Atomic classes and CAS (lock-free)

An `AtomicInteger` wraps a number with hardware-level atomic operations. The key idea is **CAS** — **Compare-And-Set**: “set the value to Y *only if* it is still X; tell me whether you succeeded.”

```

AtomicInteger stock = new AtomicInteger(100);

stock.incrementAndGet();              // atomic ++, no lock needed
stock.compareAndSet(100, 99);         // true only if value was exactly 100

// The universal CAS RETRY LOOP - decrement only while positive:
public boolean tryPurchase() {
    while (true) {
        int current = stock.get();     // 1. read
        if (current <= 0) return false; // 2. check    (sold out)
        if (stock.compareAndSet(current, current - 1))
            return true;               // 3. act - succeeds only if nobody
        // changed stock since step 1; else retry
    }
}

```

Tip: Say this in the interview: “CAS is optimistic — instead of blocking, I attempt the update and retry on conflict. Under low contention it's faster than locking.”

3.6 Tool #5 - ConcurrentHashMap and its atomic compound operations

```

Map<String, TokenBucket> buckets = new ConcurrentHashMap<>();

// WRONG - check-then-act race across two calls:
if (!buckets.containsKey(user)) buckets.put(user, new TokenBucket());

// RIGHT - one atomic call; exactly one bucket is ever created per user:

```



```

buckets.computeIfAbsent(user, u -> new TokenBucket());

// Other atomic compound ops worth knowing:
map.putIfAbsent(k, v);           // insert only if absent
map.merge(k, 1, Integer::sum);   // atomic "add to counter"
map.compute(k, (key, old) -> ...); // atomic read-modify-write for one key
map.remove(k, expectedValue);    // remove only if value still matches

```

- Never lock a whole `ConcurrentHashMap`; per-key atomic ops are the whole point (internally it locks tiny bins, and reads don't block at all).
- Versus `Collections.synchronizedMap/Hashtable`: those take ONE global lock for every call — correct but a bottleneck, and compound operations still race without external locking.
- `CopyOnWriteArrayList` suits read-mostly lists (listeners, parking spots) — every write copies the array, reads are lock-free.

3.7 Tool #6 - BlockingQueue: the producer-consumer backbone

```

BlockingQueue<String> queue = new LinkedBlockingQueue<>(10_000);

queue.put(msg);           // producer: BLOCKS if the queue is full (back-pressure)
queue.take();             // consumer: BLOCKS until an item arrives (no busy-waiting)
queue.offer(msg);          // non-blocking variant: returns false when full (drop policy)

```

This one class removes all manual `wait()/notify()` code. It powers the `async Logger` (Problem 10) and the `Task Scheduler` (Problem 11).

3.8 Coarse-grained vs fine-grained locking

Approach	Example	Trade-off
Coarse-grained: one lock for the whole service	<code>synchronized</code> on <code>BookingService</code> — every booking waits in one line.	Trivially correct, easy to reason about, but zero parallelism. Fine as a first version — say you'd refine it.
Fine-grained: one lock per resource	Each <code>Seat/spot</code> has its own lock; users booking different seats never block each other.	Scales beautifully; costs more care (multiple locks -> deadlock risk, see 3.10).

Tip: The magic sentence interviewers love: “I'll start coarse-grained for correctness, then move the lock down to per-seat granularity so independent bookings don't serialize.”

3.9 Optimistic vs pessimistic locking

Pessimistic: assume conflict, take the lock before touching data (`synchronized`, `ReentrantLock`; in databases: `SELECT ... FOR UPDATE`). Best when contention is high — e.g. a hot seat during a blockbuster's booking rush.

Optimistic: assume no conflict, work on a copy, and at commit time verify nobody changed the data (CAS, or a **version number**; in databases: a version column). Best when conflicts are rare — retry on failure.

Optimistic locking with a version number - the in-memory twin of a DB version column

```

class VersionedWallet {
    private double balance;
    private int version;           // bumps on every successful write

    public synchronized double[] read() { // returns {balance, version}
        return new double[]{balance, version};
    }
    public synchronized boolean tryUpdate(double newBalance, int expectedVersion) {
        if (version != expectedVersion) return false; // someone got there first
        balance = newBalance;
        version++;
        return true; // caller retries on false
    }
}

```

3.10 Deadlock: how it happens, how you prevent it

Deadlock: thread A holds lock 1 and wants lock 2; thread B holds lock 2 and wants lock 1. Both wait forever. It requires holding one lock while acquiring another — so the fixes all attack that.

```
// Money transfer without deadlock: ALWAYS acquire locks in one GLOBAL ORDER.
void transfer(Account a, Account b, double amount) {
    Account first = a.id.compareTo(b.id) < 0 ? a : b;    // lower id locked first,
    Account second = a.id.compareTo(b.id) < 0 ? b : a;    // everywhere in the codebase
    synchronized (first) {
        synchronized (second) {
            a.debit(amount);
            b.credit(amount);
        }
    }
}
```

- **Lock ordering** (above): impose a global order (sort by id) — circular waiting becomes impossible.
- **tryLock with timeout**: if you can't get the second lock in time, release everything and retry — used in Problem 2's seat holds.
- **Hold one lock at a time**: restructure so you never nest locks. The best fix when you can manage it.

3.11 Thread-safe Singleton - the three acceptable answers

```
// 1) Eager: thread-safe via class loading. Fine when construction is cheap.
class ServiceA {
    private static final ServiceA INSTANCE = new ServiceA();
    private ServiceA() {}
    public static ServiceA getInstance() { return INSTANCE; }
}

// 2) Holder idiom: LAZY and thread-safe with zero locks. (Recommended)
class ServiceB {
    private ServiceB() {}
    private static class Holder { static final ServiceB INSTANCE = new ServiceB(); }
    public static ServiceB getInstance() { return Holder.INSTANCE; }
}

// 3) Double-checked locking: classic follow-up. `volatile` is MANDATORY -
// without it, another thread can observe a half-constructed object.
class ServiceC {
    private static volatile ServiceC instance;
    private ServiceC() {}
    public static ServiceC getInstance() {
        if (instance == null) {                                // fast path: no lock once built
            synchronized (ServiceC.class) {
                if (instance == null) instance = new ServiceC();
            }
        }
        return instance;
    }
}
```

3.12 The golden rules card

- Keep state private; mutate it only through methods that guard it. Never leak a mutable collection from a getter (return a copy or unmodifiable view).
- Immutable objects (final fields, no setters) need no locks at all — make Ticket, Split, LogEvent immutable.
- Lock the check **and** the act together; keep critical sections tiny; no I/O, no callbacks, no external calls while holding a lock.
- volatile gives **visibility** (readers see the latest write) but NOT atomicity — volatile int count; count++ is still a race. Use it for status flags, not counters.
- Prefer, in order: immutability -> concurrent collections/atomics -> synchronized -> ReentrantLock. Reach for the simplest tool that works.
- Prove it in the demo: hammer the service from an ExecutorService and assert the invariant (sold == 100, never 101).

4. Solved Problems

Each problem follows the same rhythm you should use in the interview: **requirements to clarify** -> **design** -> **code** -> **concurrency notes** -> **follow-ups**. The code is compact on purpose — short enough to type in an hour, complete enough to run.

PROBLEM 1

Medium | ~50 min

Parking Lot

The classic opener | Patterns: Strategy, Singleton | Concurrency: CAS per spot

Requirements to clarify (write these as comments)

- Vehicle types: bike, car, truck; each parks only in a matching spot type.
- Operations: park (returns a ticket), unpark (frees spot, returns fee).
- Fee: hourly, per vehicle type; partial hours — agree on rounding (here: round up, minimum 1 hour).
- Multiple entry gates -> **concurrent parking requests** must never assign one spot to two vehicles.

Design

Entities: Vehicle, ParkingSpot, Ticket. Service: ParkingLot (Singleton). Pricing behind FeeStrategy (Strategy) so “weekend pricing” is a new class, not an edit. The concurrency trick: each spot carries an AtomicBoolean — claiming a spot is a single CAS, so no global lock is needed to park.

Complete solution - thread-safe without a single explicit lock

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicBoolean;

enum VehicleType { BIKE, CAR, TRUCK }

class Vehicle {
    final String number;
    final VehicleType type;
    Vehicle(String number, VehicleType type) { this.number = number; this.type = type; }
}

class ParkingSpot {
    final String id;
    final VehicleType type;
    private final AtomicBoolean occupied = new AtomicBoolean(false);
    private volatile Vehicle vehicle;

    ParkingSpot(String id, VehicleType type) { this.id = id; this.type = type; }

    // Compare-And-Set: of N threads trying, exactly ONE flips false -> true.
    boolean tryOccupy(Vehicle v) {
        if (occupied.compareAndSet(false, true)) { this.vehicle = v; return true; }
        return false;
    }

    void free() { this.vehicle = null; occupied.set(false); }
}

class Ticket {
    final String id = UUID.randomUUID().toString();
    final Vehicle vehicle;
    final ParkingSpot spot;
    final long entryTimeMs = System.currentTimeMillis();
    Ticket(Vehicle vehicle, ParkingSpot spot) { this.vehicle = vehicle; this.spot = spot; }
}

interface FeeStrategy {
    double calculate(VehicleType type, long hours);
}
```

```

class HourlyFeeStrategy implements FeeStrategy {
    private static final Map<VehicleType, Double> RATE = Map.of(
        VehicleType.BIKE, 10.0, VehicleType.CAR, 30.0, VehicleType.TRUCK, 50.0);
    public double calculate(VehicleType type, long hours) {
        return Math.max(1, hours) * RATE.get(type);    // minimum 1 hour
    }
}

class ParkingLot {
    private static final ParkingLot INSTANCE = new ParkingLot();    // eager Singleton
    public static ParkingLot getInstance() { return INSTANCE; }
    private ParkingLot() {}

    private final Map<VehicleType, List<ParkingSpot>> spotsByType = new ConcurrentHashMap<>();
    private final Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();
    private final FeeStrategy feeStrategy = new HourlyFeeStrategy();

    public void addSpot(ParkingSpot spot) {
        // CopyOnWriteArrayList: spots are read constantly, added rarely.
        spotsByType.computeIfAbsent(spot.type, t -> new CopyOnWriteArrayList<>()).add(spot);
    }

    public Ticket park(Vehicle v) {
        for (ParkingSpot spot : spotsByType.getDefault(v.type, List.of())) {
            if (spot.tryOccupy(v)) {    // atomic claim, no global lock
                Ticket t = new Ticket(v, spot);
                activeTickets.put(t.id, t);
                return t;
            }
        }
        throw new IllegalStateException("No free spot for " + v.type);
    }

    public double unpark(String ticketId) {
        Ticket t = activeTickets.remove(ticketId);    // atomic: a ticket unparks once
        if (t == null) throw new IllegalArgumentException("Invalid or already-used ticket");
        long hours = (System.currentTimeMillis() - t.entryTimeMs) / 3_600_000;
        t.spot.free();
        return feeStrategy.calculate(t.vehicle.type, hours);
    }
}

public class Main {
    public static void main(String[] args) {
        ParkingLot lot = ParkingLot.getInstance();
        lot.addSpot(new ParkingSpot("C1", VehicleType.CAR));
        lot.addSpot(new ParkingSpot("C2", VehicleType.CAR));

        Ticket t1 = lot.park(new Vehicle("KA-01-1234", VehicleType.CAR));
        System.out.println("Parked at " + t1.spot.id);
        System.out.println("Fee on exit: Rs. " + lot.unpark(t1.id));
        lot.park(new Vehicle("KA-02-9999", VehicleType.CAR));
        lot.park(new Vehicle("KA-03-0007", VehicleType.CAR));
        try { lot.park(new Vehicle("KA-04-1111", VehicleType.CAR)); }
        catch (IllegalStateException e) { System.out.println("Expected: " + e.getMessage()); }
    }
}

```

Concurrency notes (say these out loud)

- Two gates parking simultaneously race for the same spot; `occupied.compareAndSet(false, true)` guarantees one winner — the loser simply tries the next spot.
- `activeTickets.remove(ticketId)` is atomic, so the same ticket can never be unparked (and refunded) twice.
- The vehicle field is `volatile` so the exit gate's thread sees the vehicle written by the entry gate's thread.

Follow-ups to expect

- Nearest-spot / floor-wise allocation -> extract `SpotAllocationStrategy` (Strategy again).
- Display board of free counts per type -> `AtomicInteger` counters or an `Observer` on park/unpark events.
- Electric spots with chargers -> subclass or a feature enum on `ParkingSpot` — discuss composition vs inheritance.

Movie Ticket Booking (BookMyShow)

THE concurrency showpiece | Fine-grained locks, two-phase booking, rollback

Requirements to clarify

- A show has seats; users can view availability, **hold** seats while paying, then **confirm**.
- A hold expires after N minutes (payment window) and the seats auto-release.
- Two users clicking the same seat at the same moment: **exactly one** gets it — the core requirement.
- Multi-seat selection is all-or-nothing: never leave a user with 2 of the 3 seats they wanted.

Design

Seat lifecycle is a tiny state machine: AVAILABLE -> HELD -> BOOKED (with HELD able to expire back to AVAILABLE). Each Seat guards its own transitions with synchronized — **fine-grained locking**, so bookings for different seats run fully in parallel. Multi-seat holds acquire seats one at a time in **sorted order** and roll back on the first failure — all-or-nothing without ever nesting locks (so deadlock is structurally impossible).

Complete solution - the demo literally races two threads for the same seat

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

enum SeatStatus { AVAILABLE, HELD, BOOKED }

class Seat {
    final String id;
    private SeatStatus status = SeatStatus.AVAILABLE;
    private String heldByUser;
    private long holdExpiryMs;

    Seat(String id) { this.id = id; }

    // Every state transition is synchronized on THIS seat only (fine-grained).
    synchronized boolean tryHold(String userId, long holdMillis) {
        releaseIfExpired();
        if (status != SeatStatus.AVAILABLE) return false;
        status = SeatStatus.HELD;
        heldByUser = userId;
        holdExpiryMs = System.currentTimeMillis() + holdMillis;
        return true;
    }

    synchronized boolean confirm(String userId) {
        releaseIfExpired();
        boolean ok = status == SeatStatus.HELD && userId.equals(heldByUser);
        if (ok) status = SeatStatus.BOOKED;
        return ok;
    }

    synchronized void release(String userId) {
        if (status == SeatStatus.HELD && userId.equals(heldByUser)) {
            status = SeatStatus.AVAILABLE;
            heldByUser = null;
        }
    }

    // Lazy expiry: checked on every touch - no background thread required.
    private void releaseIfExpired() {
        if (status == SeatStatus.HELD && System.currentTimeMillis() > holdExpiryMs) {
            status = SeatStatus.AVAILABLE;
            heldByUser = null;
        }
    }
}

class Show {
    final String id;
```

```

private final Map<String, Seat> seats = new ConcurrentHashMap<>();

Show(String id, int seatCount) {
    this.id = id;
    for (int i = 1; i <= seatCount; i++) seats.put("S" + i, new Seat("S" + i));
}
Seat seat(String seatId) {
    Seat s = seats.get(seatId);
    if (s == null) throw new IllegalArgumentException("No such seat: " + seatId);
    return s;
}
}

class BookingService {
    private static final long HOLD_MS = 5 * 60 * 1000; // 5-minute payment window

    // Phase 1: hold ALL requested seats or NONE.
    public boolean holdSeats(Show show, List<String> seatIds, String userId) {
        List<String> ordered = new ArrayList<>(seatIds);
        Collections.sort(ordered); // one global order = no lock "fights"
        List<Seat> acquired = new ArrayList<>();
        for (String id : ordered) {
            Seat seat = show.seat(id);
            if (seat.tryHold(userId, HOLD_MS)) {
                acquired.add(seat);
            } else {
                // someone beat us to this seat:
                for (Seat s : acquired) s.release(userId); // ROLLBACK partial holds
                return false;
            }
        }
        return true;
    }

    // Phase 2: after payment succeeds, flip HELD -> BOOKED.
    public boolean confirmBooking(Show show, List<String> seatIds, String userId) {
        return seatIds.stream().allMatch(id -> show.seat(id).confirm(userId));
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Show show = new Show("Interstellar-9PM", 20);
        BookingService svc = new BookingService();

        Runnable alice = () -> System.out.println(
            "Alice holds S1,S2: " + svc.holdSeats(show, List.of("S1", "S2"), "alice"));
        Runnable bob = () -> System.out.println(
            "Bob holds S2,S3: " + svc.holdSeats(show, List.of("S2", "S3"), "bob"));

        Thread t1 = new Thread(alice), t2 = new Thread(bob);
        t1.start(); t2.start(); t1.join(); t2.join();
        // Overlapping on S2 -> exactly one prints true; the loser's partial hold
        // (if any) was rolled back, so their other seat is free again.
        System.out.println(
            "Alice confirms: " + svc.confirmBooking(show, List.of("S1", "S2"), "alice"));
    }
}

```

Concurrency notes

- One lock **per seat**: users booking different seats never wait on each other. A single service-wide lock would also be correct — say why you chose finer granularity.
- We never hold seat A's lock while acquiring seat B's (each `tryHold` completes independently) — deadlock cannot occur; sorting plus rollback gives all-or-nothing semantics instead.
- Hold-with-expiry is the **two-phase booking** used by real ticketing systems; lazy expiry on access avoids a cleaner thread (mention a `ScheduledExecutorService` sweep as the production upgrade).
- Database mapping, if asked: pessimistic = `SELECT ... FOR UPDATE` on the seat row; optimistic = a version column with retry.

Follow-ups to expect

- Seat pricing tiers -> Strategy; theatre/screen/show hierarchy -> plain composition.

- Notify waiting users when a hold expires -> Observer.
- Scale across servers -> the JVM lock no longer protects you; you'd need a distributed lock (Redis) or DB row locks — know the sentence, don't build it.

Jaydip

PROBLEM 3

Thread-Safe LRU Cache

DSA meets design | HashMap + doubly linked list, one lock done right

Requirements to clarify

- Fixed capacity; get and put in O(1).
- On overflow, evict the **least recently used** entry; both get and put count as “use”.
- Thread-safe: many readers and writers.

Design

The classic pair: a HashMap for O(1) lookup plus a **doubly linked list** ordered by recency (head = most recent, tail = least). The two structures must change **together**, so a ConcurrentHashMap alone cannot save you — one ReentrantLock around each operation keeps the pair consistent. Dummy head/tail nodes remove every null check.

Complete solution - O(1) operations, one lock keeping map + list in sync

```
import java.util.*;
import java.util.concurrent.locks.ReentrantLock;

class LRUCache<K, V> {
    private class Node {
        K key; V value; Node prev, next;
        Node(K k, V v) { key = k; value = v; }
    }

    private final int capacity;
    private final Map<K, Node> map = new HashMap<>();
    private final Node head = new Node(null, null); // dummy: most-recent side
    private final Node tail = new Node(null, null); // dummy: least-recent side
    private final ReentrantLock lock = new ReentrantLock();

    LRUCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity must be > 0");
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        lock.lock();
        try {
            Node n = map.get(key);
            if (n == null) return null;
            moveToFront(n); // touching = recently used
            return n.value;
        } finally { lock.unlock(); } // unlock in finally, always
    }

    public void put(K key, V value) {
        lock.lock();
        try {
            Node n = map.get(key);
            if (n != null) { n.value = value; moveToFront(n); return; }
            if (map.size() == capacity) { // evict LRU = node before tail
                Node lru = tail.prev;
                unlink(lru);
                map.remove(lru.key);
            }
            Node fresh = new Node(key, value);
            map.put(key, fresh);
            addToFront(fresh);
        } finally { lock.unlock(); }
    }

    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addToFront(Node n) {
        n.next = head;
        head.prev = n;
        n.prev = null;
        n.next.prev = n;
    }
}
```



```
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
    private void moveToFront(Node n) { unlink(n); addToFront(n); }
}
```

Concurrency notes

- Why one lock and not ConcurrentHashMap? Because every operation mutates **two** structures; per-key atomicity can't keep the list consistent with the map. Naming this trade-off is the senior signal.
- Even reads take the lock — get reorders the list, so it's a write in disguise (which also rules out a plain read-write lock).
- Scaling answers: shard into N independent LRU segments by $\text{hash}(\text{key}) \% N$ (striped locking), or say “in production I'd use Caffeine, which is a segmented, lock-amortized LRU.”

Follow-ups to expect

- Add TTL per entry -> store expiry in Node, check on get (see Problem 6).
- LFU instead of LRU -> frequency buckets; describe, don't code.
- Quick alternative if time is short: LinkedHashMap with `accessOrder=true` and `removeEldestEntry` override, wrapped in synchronized methods — mention it, then build the manual version anyway (that's what they're testing).

PROBLEM 4

Rate Limiter (Token Bucket)

Backend favorite | ConcurrentHashMap.computeIfAbsent + per-bucket lock

Requirements to clarify

- Limit each user/API-key to N requests per second, with small bursts allowed.
- `allowRequest(userId)` returns `true/false` instantly (non-blocking).
- Many users, heavy concurrency; per-user isolation.

Design

Token bucket: each user owns a bucket holding up to capacity tokens that refills continuously at `refillPerSecond`. A request consumes one token; an empty bucket means 429. Burst-friendly (a full bucket absorbs a spike) and $O(1)$ memory per user. Refill lazily on each request — no timer thread needed.

Complete solution - two concurrency idioms in ~60 lines

```
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

class TokenBucket {
    private final int capacity;
    private final double refillPerSecond;
    private double tokens;
    private long lastRefillNanos;

    TokenBucket(int capacity, double refillPerSecond) {
        this.capacity = capacity;
        this.refillPerSecond = refillPerSecond;
        this.tokens = capacity;           // start full: allow a burst
        this.lastRefillNanos = System.nanoTime();
    }

    // One user's requests serialize on THEIR bucket only - different users
    // never contend (fine-grained locking again).
    synchronized boolean tryConsume() {
        refill();
        if (tokens >= 1.0) { tokens -= 1.0; return true; }
        return false;
    }

    private void refill() {
        long now = System.nanoTime();
        double elapsedSec = (now - lastRefillNanos) / 1_000_000_000.0;
        tokens = Math.min(capacity, tokens + elapsedSec * refillPerSecond);
        lastRefillNanos = now;
    }
}

class RateLimiter {
    private final Map<String, TokenBucket> buckets = new ConcurrentHashMap<>();
    private final int capacity;
    private final double refillPerSecond;

    RateLimiter(int capacity, double refillPerSecond) {
        this.capacity = capacity;
        this.refillPerSecond = refillPerSecond;
    }

    public boolean allowRequest(String userId) {
        // computeIfAbsent is ATOMIC: two first-requests from one user can never
        // create two buckets (the check-then-act is done for us, safely).
        return buckets
            .computeIfAbsent(userId, id -> new TokenBucket(capacity, refillPerSecond))
            .tryConsume();
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        RateLimiter limiter = new RateLimiter(3, 1.0);    // burst 3, refill 1/sec
        for (int i = 1; i <= 5; i++)
            System.out.println("req " + i + " -> " + limiter.allowRequest("jaydip"));
        // true, true, true, false, false (bucket refills as time passes)
    }
}

```

Concurrency notes

- Two idioms on display: **atomic bucket creation** via `computeIfAbsent` and **per-bucket synchronized** for refill+consume (they must be one atomic step, otherwise two threads both see 0.9 tokens, both refill, both pass).
- Keep the factory lambda cheap and side-effect free — the map may invoke it while holding an internal bin lock.

Follow-ups to expect

- Other algorithms — know the one-liners: **fixed window** (counter per minute; bursty at edges), **sliding window log** (store timestamps, exact but memory-heavy), **sliding window counter** (weighted blend; good compromise).
- Make algorithms swappable -> `RateLimitStrategy` interface (Strategy).
- Distributed rate limiting -> counters move to Redis; the check-then-act must become a Lua script or INCR-with-TTL, because JVM locks don't cross servers.

PROBLEM 5

Flash-Sale Inventory (Never Oversell)

The purest concurrency question | CAS retry loop + stress-test demo template

Requirements to clarify

- N units in stock; thousands of concurrent buy requests.
- Sell **exactly** N — never oversell, never undersell.

Design & code

Complete solution - memorize this `main()`: it is your reusable concurrency stress test

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

class InventoryService {
    private final AtomicInteger stock;

    InventoryService(int initialStock) { this.stock = new AtomicInteger(initialStock); }

    // WRONG (the trap they hope you'll write):
    // if (stock.get() > 0) { stock.decrementAndGet(); return true; }
    // Both threads can pass the check when stock == 1 -> stock becomes -1.

    public boolean tryPurchase() {
        while (true) { // CAS retry loop (Sec 3.5)
            int current = stock.get();
            if (current <= 0) return false; // sold out
            if (stock.compareAndSet(current, current - 1)) return true;
            // CAS failed = someone else bought in between; re-read and retry.
        }
    }

    public int remaining() { return stock.get(); }
}

public class Main {
    public static void main(String[] args) throws Exception {
        InventoryService inv = new InventoryService(100);
        AtomicInteger sold = new AtomicInteger();

        ExecutorService pool = Executors.newFixedThreadPool(50);
        for (int i = 0; i < 1_000; i++) // 1000 buyers, 100 items
            pool.execute(() -> { if (inv.tryPurchase()) sold.incrementAndGet(); });

        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("Sold: " + sold.get() + ", left: " + inv.remaining());
        // Prints "Sold: 100, left: 0" EVERY time. With the wrong version it wouldn't.
    }
}
```

Why this problem punches above its weight

- It is the smallest possible stage for the check-then-act story, the CAS retry loop, and a stress-test demo — reuse all three in bigger problems.
- Alternative accepted answer: a synchronized `tryPurchase()` — correct, simpler, but serializes all buyers; CAS lets non-conflicting attempts proceed without blocking. Saying that comparison is the point.
- Multi-SKU version: `ConcurrentHashMap<String, AtomicInteger>` with the same loop per SKU, or `map.compute(sku, (k, v) -> v > 0 ? v - 1 : v)` with a captured result.

PROBLEM 6

In-Memory Key-Value Store with TTL

Mini-Redis | Lazy + active expiry, atomic remove(key, value), daemon cleaner

Requirements to clarify

- put(key, value, ttlMillis) (ttl <= 0 means never expires), get(key), delete(key).
- Expired keys must never be returned; memory should be reclaimed eventually.
- Concurrent readers and writers.

Design

Wrap each value in an immutable Entry carrying its expiry timestamp. Expire two ways, exactly like Redis: **lazily** on read (correctness — never serve stale data) and **actively** via a background sweep (memory — reclaim keys nobody reads). The subtle line is map.remove(key, entry): remove only if the value is *still that entry*, so the cleaner can never delete a value that was just overwritten.

Complete solution - immutable entries make the whole thing almost lock-free

```
import java.util.concurrent.*;

class KVStore<K, V> {
    private static class Entry<V> {
        final V value;
        final long expiryMs;                // Long.MAX_VALUE = never expires
        Entry(V value, long expiryMs) { this.value = value; this.expiryMs = expiryMs; }
        boolean expired() { return System.currentTimeMillis() > expiryMs; }
    }

    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
    private final ScheduledExecutorService cleaner =
        Executors.newSingleThreadScheduledExecutor(r -> {
            Thread t = new Thread(r, "kv-cleaner");
            t.setDaemon(true);                // never blocks JVM shutdown
            return t;
        });

    KVStore() { // ACTIVE expiry: sweep once per second
        cleaner.scheduleAtFixedRate(
            () -> map.entrySet().removeIf(e -> e.getValue().expired()),
            1, 1, TimeUnit.SECONDS);
    }

    public void put(K key, V value, long ttlMillis) {
        long expiry = ttlMillis <= 0 ? Long.MAX_VALUE
            : System.currentTimeMillis() + ttlMillis;
        map.put(key, new Entry<>(value, expiry));
    }

    public V get(K key) { // LAZY expiry: never serve stale
        Entry<V> e = map.get(key);
        if (e == null) return null;
        if (e.expired()) {
            map.remove(key, e); // atomic remove(k, v): only if value is STILL e -
            return null;        // a concurrent fresh put() is never destroyed
        }
        return e.value;
    }

    public void delete(K key) { map.remove(key); }
}

public class Main {
    public static void main(String[] args) throws Exception {
        KVStore<String, String> store = new KVStore<>();
        store.put("otp:jaydip", "482913", 1000); // 1s TTL
        System.out.println("t=0ms -> " + store.get("otp:jaydip")); // 482913
        Thread.sleep(1200);
    }
}
```

```
        System.out.println("t=1.2s -> " + store.get("otp:jaydip")); // null
    }
}
```

Concurrency notes

- Entries are **immutable**; a `put()` swaps the whole entry atomically, so readers see either the old or the new one — never a half-updated mix.
- The two-argument `remove(key, expectedValue)` is CAS-in-disguise; the one-argument `remove` would race with concurrent puts.
- Follow-ups: `getAndDelete` -> `map.remove(key)` returns the old entry atomically; per-key update -> `map.compute`; snapshotting, LRU eviction on top (Problem 3).

Jaydip

Splitwise (Expense Sharing)

Strategy pattern showcase | Balance netting logic interviewers probe

Requirements to clarify

- Add an expense: who paid, total amount, participants, split type (EQUAL / EXACT / PERCENT).
- Show who owes whom, netted (if A owes B 100 and B owes A 30, show A owes B 70).
- Validate exact amounts / percentages sum correctly.

Design

Split computation varies by type -> a SplitStrategy interface with one implementation per type (Open/Closed in action: PERCENT later = one new class). ExpenseService owns the balance sheet: balances.get(a).get(b) = how much a owes b, with reverse debts netted on every write so the sheet stays minimal.

Complete solution - three strategies plus netted balances

```
import java.util.*;

class Split {
    final String userId;
    final double amount;
    Split(String userId, double amount) { this.userId = userId; this.amount = amount; }
}

interface SplitStrategy {
    List<Split> computeSplits(double total, List<String> users, List<Double> values);
}

class EqualSplit implements SplitStrategy {
    public List<Split> computeSplits(double total, List<String> users, List<Double> values) {
        double share = Math.round(total * 100.0 / users.size()) / 100.0;
        List<Split> splits = new ArrayList<>();
        for (String u : users) splits.add(new Split(u, share));
        return splits;
    }
}

class ExactSplit implements SplitStrategy {
    public List<Split> computeSplits(double total, List<String> users, List<Double> values) {
        double sum = 0;
        for (double v : values) sum += v;
        if (Math.abs(sum - total) > 0.01)
            throw new IllegalArgumentException("Exact amounts must add up to total");
        List<Split> splits = new ArrayList<>();
        for (int i = 0; i < users.size(); i++)
            splits.add(new Split(users.get(i), values.get(i)));
        return splits;
    }
}

class PercentSplit implements SplitStrategy {
    public List<Split> computeSplits(double total, List<String> users, List<Double> values) {
        double sum = 0;
        for (double v : values) sum += v;
        if (Math.abs(sum - 100.0) > 0.01)
            throw new IllegalArgumentException("Percentages must add up to 100");
        List<Split> splits = new ArrayList<>();
        for (int i = 0; i < users.size(); i++)
            splits.add(new Split(users.get(i), total * values.get(i) / 100.0));
        return splits;
    }
}

class ExpenseService {
    // balances.get(a).get(b) = amount a owes b (only one direction ever stored)
    private final Map<String, Map<String, Double>> balances = new HashMap<>();
}
```

```

public synchronized void addExpense(String paidBy, double total, List<String> users,
                                   SplitStrategy strategy, List<Double> values) {
    for (Split split : strategy.computeSplits(total, users, values)) {
        if (split.userId.equals(paidBy)) continue; // payer owes nothing to self
        addBalance(split.userId, paidBy, split.amount);
    }
}

// Net out any reverse debt first, so the sheet never shows both directions.
private void addBalance(String from, String to, double amount) {
    double reverse = balances.getOrDefault(to, Map.of()).getOrDefault(from, 0.0);
    if (reverse > 0) {
        double settled = Math.min(reverse, amount);
        balances.get(to).put(from, reverse - settled);
        amount -= settled;
    }
    if (amount > 0)
        balances.computeIfAbsent(from, k -> new HashMap<>())
            .merge(to, amount, Double::sum);
}

public synchronized void showBalances() {
    balances.forEach((from, m) -> m.forEach((to, amt) -> {
        if (amt > 0.009) System.out.printf("%s owes %s: %.2f%n", from, to, amt);
    }));
}

public class Main {
    public static void main(String[] args) {
        ExpenseService svc = new ExpenseService();
        List<String> trio = List.of("alice", "bob", "cara");

        svc.addExpense("alice", 900, trio, new EqualSplit(), null); // 300 each
        svc.addExpense("bob", 300, List.of("alice", "bob"),
                       new ExactSplit(), List.of(120.0, 180.0));
        svc.showBalances();
        // bob owes alice: 180.00 (300 - 120 netted)
        // cara owes alice: 300.00
    }
}

```

Notes to mention out loud

- Money in double is fine for an interview, but say it: “production would use `BigDecimal` or integer paise to avoid floating-point drift.”
- Equal split rounding leaves a stray paise (100/3) — assign the remainder to the payer or first user; mentioning this earns real points.
- `addExpense` is synchronized because one expense mutates several inner maps; they must change as one atomic unit (same reasoning as the LRU cache).

Follow-ups to expect

- Simplify/settle-up (minimum number of transactions) -> compute each user's net, then greedily match largest creditor with largest debtor. Describe the algorithm; code only if asked.
- Groups, expense history, and an `Expense` entity with `id/notes` -> straightforward extensions of this skeleton.

Snake & Ladder

Game-design template | One Map for snakes AND ladders, clean turn rotation

Requirements to clarify

- Board size (default 100), configurable snakes and ladders, 2+ players.
- Exact-landing rule on the final cell? (Here: overshoot = skip the move.)
- Dice: single die 1-6; design so a “crooked dice” variant is easy.

Design

The elegant trick: snakes and ladders are the **same concept** — “landing on X moves you to Y” — so one `Map<Integer, Integer>` of jumps handles both. Turn order is a deque rotating players. Dice sits behind its own tiny class so a loaded or two-dice variant is a one-line swap (Strategy without the ceremony).

Complete solution - runs a full game with commentary

```
import java.util.*;

class Board {
    final int size;
    private final Map<Integer, Integer> jumps = new HashMap<>(); // snakes AND ladders

    Board(int size) { this.size = size; }

    void addSnake(int head, int tail) {
        if (tail >= head) throw new IllegalArgumentException("snake must go DOWN");
        jumps.put(head, tail);
    }
    void addLadder(int start, int end) {
        if (end <= start) throw new IllegalArgumentException("ladder must go UP");
        jumps.put(start, end);
    }
    int resolve(int position) { return jumps.getOrDefault(position, position); }
}

class Dice {
    private final Random random = new Random();
    int roll() { return random.nextInt(6) + 1; }
}

class Player {
    final String name;
    int position = 0;
    Player(String name) { this.name = name; }
}

class Game {
    private final Board board;
    private final Dice dice = new Dice();
    private final Deque<Player> turns = new ArrayDeque<>();

    Game(Board board, List<Player> players) {
        if (players.size() < 2) throw new IllegalArgumentException("need 2+ players");
        this.board = board;
        turns.addAll(players);
    }

    void play() {
        while (true) {
            Player p = turns.pollFirst();
            int roll = dice.roll();
            int target = p.position + roll;
            if (target <= board.size) { // overshoot = skip move
                int before = target;
                p.position = board.resolve(target);
                System.out.printf("%s rolled %d -> %d%s\n", p.name, roll, p.position,
```

```

        p.position != before ? " (jump!)" : "");
    } else {
        System.out.printf("%s rolled %d -> needs exact, stays at %d\n",
            p.name, roll, p.position);
    }
    if (p.position == board.size) {
        System.out.println(p.name + " WINS!");
        return;
    }
    turns.addLast(p); // rotate turn
}
}

public class Main {
    public static void main(String[] args) {
        Board board = new Board(100);
        board.addLadder(4, 25); board.addLadder(21, 82);
        board.addSnake(99, 12); board.addSnake(56, 3);
        new Game(board, List.of(new Player("Alice"), new Player("Bob"))).play();
    }
}

```

Notes & follow-ups

- Concurrency deliberately absent: a turn-based game is sequential by nature — **saying that** is better than sprinkling locks. If asked about many simultaneous game rooms: one single-threaded Game per room; rooms in a ConcurrentHashMap.
- Classic follow-ups: roll-again-on-six, exact-landing toggle, multiple dice (sum in Dice), snake-bite penalties — each lands in exactly one class if the design above is followed.
- Same skeleton solves Tic-Tac-Toe: Board + Player rotation + a checkwinner that keeps row/column/diagonal counters for O(1) detection per move.

Vending Machine

THE State-pattern question | Illegal actions become impossible, not if-guarded

Requirements to clarify

- Flow: insert coins -> select item -> dispense + change; cancel/refund optional.
- Handle: unknown item code, out of stock, insufficient balance.
- Single physical machine -> effectively one user at a time (state pattern shines; wrap the public API in synchronized if the interviewer insists on threads).

Design

Behavior depends on the machine's current state — inserting a coin means something different when idle vs while dispensing. The **State pattern** gives each state its own class implementing the same interface; the machine delegates every action to its current state. The win: no giant if (state == ...) ladders, and adding a state (MAINTENANCE) touches nothing existing.

Complete solution - every illegal action handled by design, not by if-ladders

```
import java.util.*;

class Item {
    final String code, name;
    final int price;
    int stock;
    Item(String code, String name, int price, int stock) {
        this.code = code; this.name = name; this.price = price; this.stock = stock;
    }
}

interface State {
    void insertCoin(VendingMachine m, int amount);
    void selectItem(VendingMachine m, String code);
    void dispense(VendingMachine m);
}

class IdleState implements State {
    public void insertCoin(VendingMachine m, int amount) {
        m.addBalance(amount);
        m.setState(m.HAS_MONEY);
        System.out.println("Accepted. Balance: " + m.getBalance());
    }
    public void selectItem(VendingMachine m, String code) {
        System.out.println("Insert coins first.");
    }
    public void dispense(VendingMachine m) { System.out.println("Nothing to dispense."); }
}

class HasMoneyState implements State {
    public void insertCoin(VendingMachine m, int amount) {
        m.addBalance(amount);
        System.out.println("Balance: " + m.getBalance());
    }
    public void selectItem(VendingMachine m, String code) {
        Item item = m.getItem(code);
        if (item == null) { System.out.println("Unknown code."); return; }
        if (item.stock == 0) { System.out.println("Out of stock."); return; }
        if (m.getBalance() < item.price) {
            System.out.println("Need " + (item.price - m.getBalance()) + " more.");
            return;
        }
        m.setSelected(item);
        m.setState(m.DISPENSING);
        m.dispense();
    }
    public void dispense(VendingMachine m) { System.out.println("Select an item first."); }
}
```

```

}

class DispensingState implements State {
    public void insertCoin(VendingMachine m, int amount) { System.out.println("Busy..."); }
    public void selectItem(VendingMachine m, String code) { System.out.println("Busy..."); }
    public void dispense(VendingMachine m) {
        Item item = m.getSelected();
        item.stock--;
        int change = m.getBalance() - item.price;
        m.resetBalance();
        m.setSelected(null);
        m.setState(m.IDLE);
        System.out.println("Dispensed " + item.name + ". Change: " + change);
    }
}

class VendingMachine {
    final State IDLE = new IdleState();           // stateless -> safe to share
    final State HAS_MONEY = new HasMoneyState();
    final State DISPENSING = new DispensingState();

    private State state = IDLE;
    private int balance = 0;
    private Item selected;
    private final Map<String, Item> inventory = new HashMap<>();

    // Public API just delegates to the current state:
    public void insertCoin(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        state.insertCoin(this, amount);
    }
    public void selectItem(String code) { state.selectItem(this, code); }
    public void dispense() { state.dispense(this); }

    public void stockItem(Item item) { inventory.put(item.code, item); }

    // Package-private helpers used by states:
    void setState(State s)      { this.state = s; }
    void addBalance(int amt)    { this.balance += amt; }
    void resetBalance()         { this.balance = 0; }
    int getBalance()            { return balance; }
    Item getItem(String code)   { return inventory.get(code); }
    Item getSelected()          { return selected; }
    void setSelected(Item i)    { this.selected = i; }
}

public class Main {
    public static void main(String[] args) {
        VendingMachine vm = new VendingMachine();
        vm.stockItem(new Item("A1", "Chips", 20, 2));
        vm.selectItem("A1");           // Insert coins first.
        vm.insertCoin(10);             // Accepted. Balance: 10
        vm.selectItem("A1");           // Need 10 more.
        vm.insertCoin(20);             // Balance: 30
        vm.selectItem("A1");           // Dispensed Chips. Change: 10
    }
}

```

Notes & follow-ups

- Say the pattern's payoff explicitly: “each state class only handles what is legal in that state; adding MAINTENANCE or CANCELLING is a new class, zero edits elsewhere” — Open/Closed again.
- Follow-ups: refund/cancel (new transition to IDLE returning balance), coin denominations and change-making (greedy with a coin inventory — Chain of Responsibility variant), admin restock API.
- Same pattern answers: Elevator states, ATM states, Order lifecycle (PLACED -> PACKED -> SHIPPED -> DELIVERED).

Async Logger Framework

Producer-consumer in the wild | *BlockingQueue* + daemon worker + pluggable sinks

Requirements to clarify

- Levels `DEBUG < INFO < ERROR` with a configurable minimum level.
- Pluggable destinations (console, file, ...) — new sink without touching the logger.
- Logging must not slow callers down -> **async**: callers enqueue, a worker thread writes.

Design

Two clean seams: an Appender interface for destinations (Strategy), and a `BlockingQueue` between callers (producers) and one background worker (consumer). `queue.take()` blocks the worker when idle — zero busy-waiting, zero manual wait/notify. This is Section 3.7 turned into a product.

Complete solution - many producers, one consumer, no locks written by hand

```
import java.util.concurrent.*;

enum LogLevel { DEBUG, INFO, ERROR }

interface Appender { void append(String line); }

class ConsoleAppender implements Appender {
    public void append(String line) { System.out.println(line); }
}

class AsyncLogger {
    private final LogLevel minLevel;
    private final Appender appender;
    private final BlockingQueue<String> queue = new LinkedBlockingQueue<>(10_000);

    AsyncLogger(LogLevel minLevel, Appender appender) {
        this.minLevel = minLevel;
        this.appender = appender;
        Thread worker = new Thread(this::drain, "log-worker");
        worker.setDaemon(true); // JVM can exit without us
        worker.start();
    }

    // PRODUCER side - called from any thread, returns immediately.
    public void log(LogLevel level, String msg) {
        if (level.ordinal() < minLevel.ordinal()) return; // filtered out
        String line = System.currentTimeMillis() + " [" + level + "] ["
            + Thread.currentThread().getName() + "] " + msg;
        boolean accepted = queue.offer(line); // non-blocking; drops if full
        if (!accepted) System.err.println("LOG DROPPED (queue full)");
    }

    public void debug(String msg) { log(LogLevel.DEBUG, msg); }
    public void info(String msg) { log(LogLevel.INFO, msg); }
    public void error(String msg) { log(LogLevel.ERROR, msg); }

    // CONSUMER side - single worker thread, so appends never interleave.
    private void drain() {
        try {
            while (true) appender.append(queue.take()); // blocks until a line arrives
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        AsyncLogger log = new AsyncLogger(LogLevel.INFO, new ConsoleAppender());
        ExecutorService pool = Executors.newFixedThreadPool(4);
    }
}
```

```

    for (int i = 1; i <= 8; i++) {
        int n = i;
        pool.execute(() -> {
            log.debug("noise " + n);
            log.info("processed request " + n);
        });
    }
    pool.shutdown();
    pool.awaitTermination(2, TimeUnit.SECONDS);
    Thread.sleep(200);
}

```

Design decisions to narrate

- Bounded queue + offer = an explicit **overload policy** (drop). Alternatives: put blocks the caller (back-pressure) or drop-oldest. Naming the choice matters more than the choice.
- One consumer thread means file writes never interleave — no lock needed on the sink itself.
- Follow-ups: multiple appenders per level (keep a list, or Chain of Responsibility where each handler logs and forwards), log formatting (Strategy), graceful shutdown (a poison-pill message the worker exits on, after draining).

PROBLEM 11

Task Scheduler (Run After a Delay)

DelayQueue does the hard part | Dispatcher thread + worker pool

Requirements to clarify

- `schedule(task, delayMs)`; tasks run when due, not before.
- Many tasks may be due simultaneously -> execute on a worker pool, not the dispatcher thread.
- Recurring tasks as a follow-up.

Design

`DelayQueue` is a thread-safe priority queue whose `take()` **blocks until the head element's delay expires** — it solves “sleep until the next task is due, but wake early if an earlier one arrives” in one class. One dispatcher thread takes due tasks and hands them to an `ExecutorService` so a slow task never delays the next one. (Also say: “in production I'd reach for `ScheduledExecutorService` directly — this shows how it works inside.”)

Complete solution - scheduling correctness delegated to `DelayQueue`

```
import java.util.concurrent.*;

class ScheduledTask implements Delayed {
    final Runnable job;
    final long runAtMs;

    ScheduledTask(Runnable job, long delayMs) {
        this.job = job;
        this.runAtMs = System.currentTimeMillis() + delayMs;
    }
    public long getDelay(TimeUnit unit) {           // how long until due (<= 0 = due)
        return unit.convert(runAtMs - System.currentTimeMillis(), TimeUnit.MILLISECONDS);
    }
    public int compareTo(Delayed other) {           // earliest deadline first
        return Long.compare(runAtMs, ((ScheduledTask) other).runAtMs);
    }
}

class TaskScheduler {
    private final DelayQueue<ScheduledTask> queue = new DelayQueue<>();
    private final ExecutorService workers = Executors.newFixedThreadPool(4);

    TaskScheduler() {
        Thread dispatcher = new Thread(() -> {
            try {
                while (true) {
                    ScheduledTask task = queue.take();    // BLOCKS until head is DUE
                    workers.execute(task.job);            // never run on dispatcher
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "dispatcher");
        dispatcher.setDaemon(true);
        dispatcher.start();
    }

    public void schedule(Runnable job, long delayMs) {
        queue.put(new ScheduledTask(job, delayMs));
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        TaskScheduler scheduler = new TaskScheduler();
        long t0 = System.currentTimeMillis();
        scheduler.schedule(() -> System.out.println(
            "B ran at +" + (System.currentTimeMillis() - t0) + "ms"), 1000);
        scheduler.schedule(() -> System.out.println(
```

```
        "A ran at +" + (System.currentTimeMillis() - t0) + "ms"), 300);
        Thread.sleep(1500);           // A prints ~300ms, B ~1000ms - order by due time
    }
}
```

Follow-ups to expect

- Recurring tasks: after execution, re-enqueue with `runAtMs + intervalMs` (fixed-rate) or `now + interval` (fixed-delay) — know the difference.
- Cancel: keep a `Map<taskId, ScheduledTask>` and a cancelled flag the dispatcher checks before executing.
- Cron-style times, priorities, retries with backoff — each is a small field/strategy on `ScheduledTask`.

Jaydip

PROBLEM 12

Elevator System

Design conversation first, code second | SCAN algorithm, pluggable assignment

Requirements to clarify

- N elevators, M floors; external requests (floor + direction) and internal requests (target floor).
- Scheduling goal: don't zig-zag — serve all stops in the current direction, then reverse (the **SCAN** algorithm, exactly like disk scheduling).
- Which elevator serves an external request? Start with nearest-elevator; keep it swappable.

Design

Each Elevator keeps two sorted stop sets: floors above (visited ascending) and floors below (descending). A clock `step()` moves one floor and opens doors when the current floor is a stop. The `ElevatorController` assigns pickups via a strategy. All mutating methods are synchronized per elevator: requests arrive from many threads, and again — fine-grained, one lock per elevator, elevators never block each other.

Core solution - in the interview, spend the saved minutes narrating trade-offs

```
import java.util.*;

enum Direction { UP, DOWN, IDLE }

class Elevator {
    final int id;
    private int floor = 0;
    private Direction dir = Direction.IDLE;
    private final TreeSet<Integer> upStops = new TreeSet<>();
    private final TreeSet<Integer> downStops = new TreeSet<>(Comparator.reverseOrder());

    Elevator(int id) { this.id = id; }

    synchronized void addStop(int target) {
        if (target == floor) return;
        if (target > floor) upStops.add(target); else downStops.add(target);
        if (dir == Direction.IDLE) dir = target > floor ? Direction.UP : Direction.DOWN;
    }

    // Called every tick by a clock/scheduler thread: SCAN movement.
    synchronized void step() {
        if (dir == Direction.UP) {
            floor++;
            if (upStops.remove(floor)) System.out.println("E" + id + " opens at " + floor);
            if (upStops.isEmpty())
                dir = downStops.isEmpty() ? Direction.IDLE : Direction.DOWN;
        } else if (dir == Direction.DOWN) {
            floor--;
            if (downStops.remove(floor)) System.out.println("E" + id + " opens at " + floor);
            if (downStops.isEmpty())
                dir = upStops.isEmpty() ? Direction.IDLE : Direction.UP;
        }
    }

    synchronized int distanceTo(int f) { return Math.abs(floor - f); }
    synchronized Direction direction() { return dir; }
}

interface AssignmentStrategy {
    Elevator pick(List<Elevator> elevators, int floor);
}

class NearestElevator implements AssignmentStrategy {
    public Elevator pick(List<Elevator> elevators, int floor) {
        return elevators.stream()
            .min(Comparator.comparingInt(e -> e.distanceTo(floor)))
    }
}
```

```

        .orElseThrow();
    }
}

class ElevatorController {
    private final List<Elevator> elevators;
    private final AssignmentStrategy strategy;

    ElevatorController(List<Elevator> elevators, AssignmentStrategy strategy) {
        this.elevators = elevators;
        this.strategy = strategy;
    }
    public void requestPickup(int floor) { strategy.pick(elevators, floor).addStop(floor); }
    public void tick() { elevators.forEach(Elevator::step); }
}

public class Main {
    public static void main(String[] args) {
        List<Elevator> lifts = List.of(new Elevator(1), new Elevator(2));
        ElevatorController ctrl = new ElevatorController(lifts, new NearestElevator());
        ctrl.requestPickup(5);           // external call at floor 5
        lifts.get(0).addStop(9);         // rider inside presses 9
        for (int t = 0; t < 12; t++) ctrl.tick(); // simulate the clock
    }
}

```

What to narrate (this problem is 50% talking)

- Why two TreeSet's beat one FIFO queue: FIFO makes the lift zig-zag; SCAN serves everything en route — state the comparison explicitly.
- Better assignment than “nearest”: prefer an elevator already moving **toward** the caller in the same direction; it's one new AssignmentStrategy class — the seam you built pays off live.
- State pattern (Problem 9) is the natural refactor if the interviewer pushes on door-open/maintenance states.

4.13 Twelve More Frequently Asked Problems - Approach in One Breath

You cannot pre-code everything, but you can pre-*think* everything. Each row is the 30-second design answer that gets you moving immediately.

Problem	Core entities	The approach in one breath
Tic-Tac-Toe	Board, Player, Game	Snake & Ladder skeleton + O(1) win check: per-player row/col/diagonal counters updated on each move.
Chess (structure only)	Board, Piece hierarchy, Move	Abstract Piece with canMove(); Factory creates pieces; Game validates turns. Scope ruthlessly — movement rules for 2-3 pieces max.
ATM	ATM, Card, Account, CashDispenser	State pattern (idle/card-in/authenticated) + Chain of Responsibility for notes (2000 -> 500 -> 100).
Library Management	Book, BookCopy, Member, Loan	Separate Book (title) from BookCopy (physical unit); loans map copy to member with due dates; fines via Strategy.
Cab Booking (mini-Uber)	Rider, Driver, Trip, LocationIndex	Match nearest available driver (Strategy); driver status flips via CAS or per-driver lock so one driver never gets two trips.
Food Delivery (mini-Swiggy)	Restaurant, Menu, Order, DeliveryAgent	Order state machine (State) + Observer to notify user on status change; agent assignment mirrors cab matching.
Meeting Room Scheduler	Room, Booking, Calendar	Per-room TreeMap of start->end; a booking is valid if floorEntry and ceilingEntry don't overlap; synchronize per room.
Notification Service	Notification, Channel, UserPrefs	Channel interface (email/SMS/push = Strategy), Observer for events, retry queue via BlockingQueue.
URL Shortener (LLD flavor)	ShortUrl, CodeGenerator, Store	AtomicLong counter -> base62 encode; ConcurrentHashMap both directions; TTL via Problem 6's KVStore.
Stack Overflow (mini)	User, Question, Answer, Vote	Focus on vote invariants: one vote per user per post -> putIfAbsent on a (user, post) key; reputation via Observer.
Cricbuzz Scoreboard	Match, Innings, Over, Ball, Player	Ball is the atomic event; everything else (score, strike rotation, stats) derives from the ball stream — event-driven, easy to extend.
Chat Room (mini-Slack)	User, Room, Message	Room holds CopyOnWriteArrayList of members (read-mostly); message fan-out = Observer; history = bounded deque per room.

5. Rapid-Fire Follow-Up Questions (Concurrency & Java)

After the coding stops, the interviewer probes whether the locks you wrote were understanding or luck. These are the questions that actually get asked, with answers sized for speaking out loud.

Q1. `synchronized` vs `ReentrantLock` - when would you pick each?

`synchronized` is simpler, can't be forgotten unlocked, and is my default. `ReentrantLock` adds `tryLock` with timeout, fairness, interruptible waits, and multiple `Condition` objects - I reach for it when I need to **give up** instead of wait forever (seat holds), or need fairness. With `ReentrantLock`, `unlock()` always goes in finally.

Q2. What does `volatile` actually guarantee?

Visibility and ordering, not atomicity. A write to a `volatile` field is immediately visible to other threads (it establishes a happens-before edge). But `volatile int count; count++` is still read-modify-write, three steps, and races. Use `volatile` for status flags (`running = false`), `AtomicInteger` for counters.

Q3. Why is `ConcurrentHashMap` better than `Hashtable` / `synchronizedMap`?

Those two take one global lock for every operation - correct but fully serialized, and compound actions like check-then-put still race. `ConcurrentHashMap` locks tiny bins only on writes, reads don't block at all, and it offers atomic compound operations: `computeIfAbsent`, `merge`, `putIfAbsent`, `remove(k, v)`.

Q4. What is CAS? Any downsides?

Compare-And-Swap: a hardware instruction that sets a value only if it still equals the expected value, returning success/failure. It's optimistic and lock-free. Downsides: under heavy contention the retry loop spins wasting CPU, and the ABA problem - the value changed A to B back to A and CAS can't tell (fix: `AtomicStampedReference`, which versions the value).

Q5. Explain deadlock and the standard preventions.

Two threads each hold a lock the other needs; both wait forever. It needs four conditions simultaneously - mutual exclusion, hold-and-wait, no preemption, circular wait - so break any one: acquire locks in a single global order (sort by id), use `tryLock` with timeout and back off, or restructure to hold only one lock at a time.

Q6. Optimistic vs pessimistic locking - and where does each fit?

Pessimistic locks before touching data (`synchronized`, `SELECT FOR UPDATE`) - right when conflicts are frequent. Optimistic proceeds without a lock and validates at commit via CAS or a version number, retrying on conflict - right when conflicts are rare, since readers never block. My seat-booking hold is pessimistic per seat; my inventory counter is optimistic CAS.

Q7. `Runnable` vs `Callable`, `Future` vs `CompletableFuture`?

`Runnable` returns nothing and can't throw checked exceptions; `Callable` returns a value and can. `Future` is a handle you can only block on (`get`) or cancel; `CompletableFuture` adds non-blocking composition - `thenApply`, `thenCombine`, exception handling - so you chain work instead of parking threads.

Q8. How do you size a thread pool?

CPU-bound: about the number of cores - more threads just context-switch. I/O-bound: $\text{cores} \times (1 + \text{wait}/\text{compute})$ as a starting formula, since threads spend most time blocked, then tune under load. Always bounded pools with bounded queues - unbounded queues hide overload until you run out of memory.

Q9. Why `ExecutorService` instead of `new Thread()`?

Threads are expensive to create and unbounded thread creation can crash the process. A pool reuses threads, bounds concurrency, gives you `Futures`, graceful shutdown, and swappable policies. Rule: I create tasks, the framework owns threads.

Q10. What is a BlockingQueue and why is it beloved?

A thread-safe queue where put blocks when full and take blocks when empty - producer-consumer with back-pressure built in, no wait/notify written by hand. It decouples the speed of producers from consumers; my async logger and scheduler are both built on it.

Q11. When is ReadWriteLock the right tool?

Read-heavy shared data: many readers proceed in parallel, writers get exclusive access. Caution: if reads mutate anything (like LRU reordering on get), a 'read' is really a write and RW-locking silently breaks - that is exactly why my LRU uses one plain lock.

Q12. What's a Semaphore good for?

N permits instead of one lock: it limits concurrency rather than enforcing exclusivity. Perfect for 'at most 20 connections' or modeling limited resources - acquire() on entry, release() in finally. A parking lot IS a semaphore with capacity permits.

Q13. How does immutability help concurrency?

An object that can't change after construction can be shared by any number of threads with zero locks - there's nothing to race on. That's why my Ticket, Split, and KV Entry are final-field, no-setter classes; I mutate by replacing the object, and the reference swap is atomic.

Q14. What is ThreadLocal and one real use?

A variable with one independent copy per thread - shared name, private values, so no synchronization is needed. Classic uses: per-request context (user id, trace id) in web servers, and SimpleDateFormat before java.time, since it wasn't thread-safe. Must remove() in pools to avoid leaks.

Q15. What is a livelock? And starvation?

Livelock: threads aren't blocked but keep reacting to each other and make no progress - two people side-stepping in a corridor forever (fix: randomized backoff before retry). Starvation: a thread never gets the lock/CPU because others always win - fair locks or fair queues fix it at some throughput cost.

6. Common Mistakes That Get Candidates Rejected

X No runnable demo at the end

The single biggest reject reason. A working subset always beats a broken superset. Keep the code compiling every few minutes; freeze features at minute 50 no matter what.

X Building for 3 hours in a 1-hour round

Databases, REST controllers, UI layers, reflection-based frameworks. Everything is in-memory; a Map is your database. Say so and move on.

X The God class

One ParkingLot class doing allocation, pricing, printing and persistence. Split by responsibility - it's the difference between an SDE-1 and SDE-2 verdict.

X Everything static

Static state is a hidden global: untestable, un-mockable, and thread-unsafe by default. Instantiate services; inject strategies through constructors.

X Solving the wrong problem

Skipping the clarification phase and coding a guess. Requirements written as comments at the top of the file are both a contract and a demo script.

X Cargo-cult synchronized

Sprinkling synchronized on random methods (or nowhere). For every lock you add, be able to answer: what state does this protect, and what breaks without it?

X Swallowed exceptions and silent nulls

catch (Exception e) {} hides your bugs during the demo of all times. Fail fast with custom exceptions and messages that read well when printed.

X Magic numbers and hardcoding

5, 100, 3600000 scattered inline. Named constants or constructor parameters - takes seconds, signals craftsmanship.

X Premature patterns

Six interfaces wrapping a two-case problem. Add the seam when variation is real or hinted; otherwise name the refactor you would do.

X Going silent for the last 10 minutes

The walkthrough is scored. Practice the closing script: requirements met -> design and patterns -> concurrency story -> what I'd do next.

7. The Final 60-Minute Checklist

Run through this before saying "I'm done". Every line is a point someone lost in a real interview.

	Before you call it done
[]	Requirements written as comments at the top - and every must-have is demonstrably met.
[]	The code compiles and runs ; main() shows a happy path, a boundary case, and a failure case with a clean error message.
[]	If shared state exists: a concurrent stress test proves the invariant (exactly 100 sold; no seat double-booked).
[]	Entities, services, and strategies are separate; no God class; behavior that can vary sits behind an interface.
[]	You can answer, for every lock/atomic: <i>what does this protect, and what breaks without it?</i>
[]	Critical sections are minimal; no I/O inside locks; unlock in finally; no nested locks (or a stated lock order).
[]	Custom exceptions for invalid input; no swallowed exceptions; no nulls where an exception tells a better story.
[]	Names read like English; methods are small; no magic numbers; no dead code left behind.
[]	You can extend the likely follow-up in under 5 minutes (new vehicle type / split type / appender / state) - because a seam already exists.
[]	Walkthrough rehearsed: requirements -> design & patterns -> concurrency story -> trade-offs -> next steps.

The one-paragraph summary of this entire playbook

Clarify first and write the requirements down. Design small: entities, services, and one or two interfaces where change is likely. Build the happy path end-to-end and keep it running. Guard every piece of shared mutable state with the simplest sufficient tool - immutability, then atomics and concurrent collections, then a small synchronized block - and be ready to say what each lock protects. Prove it with a demo that includes a failure case and a concurrent stress test. Then walk the interviewer through your decisions like someone who has done this before. Because now, you have.

Tip: Practice plan: Parking Lot and BookMyShow are the highest-yield problems - code each from a blank file twice. Then LRU + Rate Limiter in one sitting (they share idioms), then one game and one State-pattern problem. Six focused sessions covers 90% of what gets asked.